1. **Aim of the practical:** Implement CRUD operation with database on NodeJS with MongoDB/MySQL

2. **Objective:** Implement CRUD operations in a Node.js backend using MongoDB/MySQL for data storage and management.

3. **Commands/Steps:**

React.js

```
 import { useState, useEffect } from "react";
import axios from "axios";
import "./App.css";

function App() {
  const [users, setUsers] = useState([]);
  const [form, setForm] = useState({ name: "", email: "", phone: "" });
  const [editId, setEditId] = useState(null);

  useEffect(() => {
    axios.get("http://localhost:5000/users").then((res) => setUsers(res.data));
  }, []);

  const handleChange = (e) => {
```

```
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const handleSubmit = async (e) => {
    e.preventDefault();
    if (editId) {
      await axios.put(`http://localhost:5000/users/${editId}`, form);
      setEditId(null);
    } else {
      await axios.post("http://localhost:5000/users", form);
    }
    setForm({ name: "", email: "", phone: "" });
    window.location.reload();
  };

  const handleEdit = (user) => {
    setForm(user);
    setEditId(user._id);
  };

  const handleDelete = async (id) => {
    await axios.delete(`http://localhost:5000/users/${id}`);
    window.location.reload();
  };

  return (
    <div className="main-container">
      <div className="form-card">
        <h2>{editId ? "Edit User" : "Add User"}</h2>
        <form onSubmit={handleSubmit}>
          <input type="text" name="name" placeholder="Enter Name" value={form.name}
onChange={handleChange} required />
```

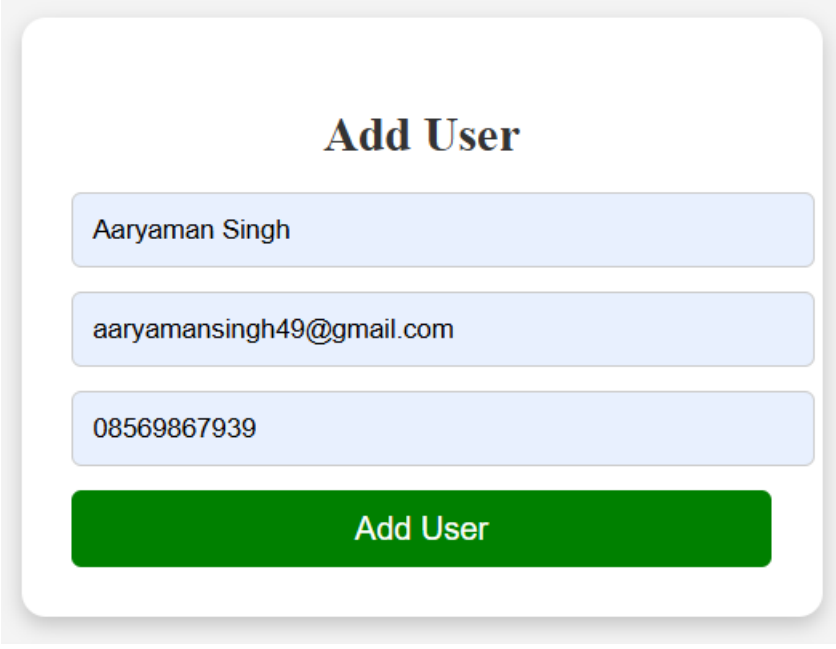
```
        <input type="email" name="email" placeholder="Enter Email" value={form.email}
onChange={handleChange} required />
        <input type="tel" name="phone" placeholder="Enter Phone" value={form.phone}
onChange={handleChange} required />
        <button type="submit">{editId ? "Update User" : "Add User"}</button>
      </form>
    </div>

    {/* Users List */}
    <div className="users-container">
      <h3>User List</h3>
      {users.length === 0 ? <p>No users found</p> : (
        <ul>
          {users.map((user) => (
            <li key={user._id}>
              <span>{user.name} - {user.email} - {user.phone}</span>
              <div>
                <button className="edit-btn" onClick={() => handleEdit(user)}>Edit</button>
                <button className="delete-btn" onClick={() =>
handleDelete(user._id)}>Delete</button>
              </div>
            </li>
          ))}
        </ul>
      )}
    </div>
  </div>
 );
}

export default App;
```

Node.js

```
const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");

const app = express();
app.use(express.json());
app.use(cors());

mongoose.connect("mongodb://localhost:27017/crudDB", {
  useNewUrlParser: true,
  useUnifiedTopology: true,
});

const UserSchema = new mongoose.Schema({
  name: String,
  email: String,
  phone: String,
});
const User = mongoose.model("User", UserSchema);

app.get("/users", async (req, res) => {
  const users = await User.find();
  res.json(users);
});

app.post("/users", async (req, res) => {
  const newUser = new User(req.body);
  await newUser.save();
  res.json({ message: "User added!" });
});
```

```
app.put("/users/:id", async (req, res) => {
  await User.findByIdAndUpdate(req.params.id, req.body);
  res.json({ message: "User updated!" });
});

app.delete("/users/:id", async (req, res) => {
  await User.findByIdAndDelete(req.params.id);
  res.json({ message: "User deleted!" });
});

app.listen(5000, () => console.log("Backend running on port 5000"));
```

4. Result:




```
_id: ObjectId('67ea2df11d76888ed9bbd737')
name : "Aaryaman Singh"
email : "aaryamansingh49@gmail.com"
phone : "08569867939"
__v : 0
```

```
_id: ObjectId('67ea2fd91d76888ed9bbd73b')
name : "Varun"
email : "varun@gmail.com"
phone : "7404729466"
__v : 0
```

5. Learning Outcomes (what I have learned):

- Understand how to integrate Node.js with MongoDB for database operations.
- Implement CRUD functionalities (Create, Read, Update, Delete) using Express.js.
- Learn to handle API requests and responses efficiently in a backend application.